

VK_RA4M2 — К а а (BG)

а а б аб а а

С а а

- За кого е този курс?
 - Какво ти трябва?
 - Как работи курсът?
 - Как компютърът "чува" и "говори"
 - Урок 0 — Първият ти звук
 - Урок 1 — DC, AC и средната точка
 - Урок 2 — Вълновите форми
 - Урок 3 — Как се генерира сигнал с числа
 - Урок 4 — Signed vs Unsigned и клипване
 - Урок 5 — Измерване: колко силен е сигналът?
 - Теоремата на Фурие
 - Урок 6 — Смесване на два сигнала
 - Урок 7 — FIR филтър
 - Урок 8 — IIR филтър
 - Урок 9 — FFT спектър
 - Урок 10 — Audiomixer
 - Урок 11 — Спектрален анализатор
 - Обобщение
 - Готови примери
 - Справочник — API
-

За а ?

Този курс е за деца и начинаещи (10+ години), които искат да разберат а а
б аб а . Не е нужно да знаеш висша математика — достатъчно е да можеш да пишеш
прости програми на Python.

Ще научиш:

- какво е звуков сигнал и как се записва с числа
- как да накараш платката да издава звук
- как да смесваш, измерваш и филтрираш сигнали
- как да видиш от какви честоти е съставен един звук

Ка б а?

- платка **VK_RA4M2**
- USB кабел и **mpremote** (за да пускаш скриптове)

- по желание: слушалки или малък усилвател с високоговорител на пин **P014**
- по желание: осцилоскоп (за да видиш вълните)

Забележка: P014 е DAC изход (аналогов). J16 е jumper за boot режим — не е звуков изход!

Ка аб ?

Всеки урок има:

1. **Об а а** — кратка теория с аналогии от реалния свят
2. **К** — готов скрипт, който пускаш на платката
3. **Наб а а** — какво трябва да чуеш / видиш
4. **Е а а** — промени 1 нещо и виж какво се случва

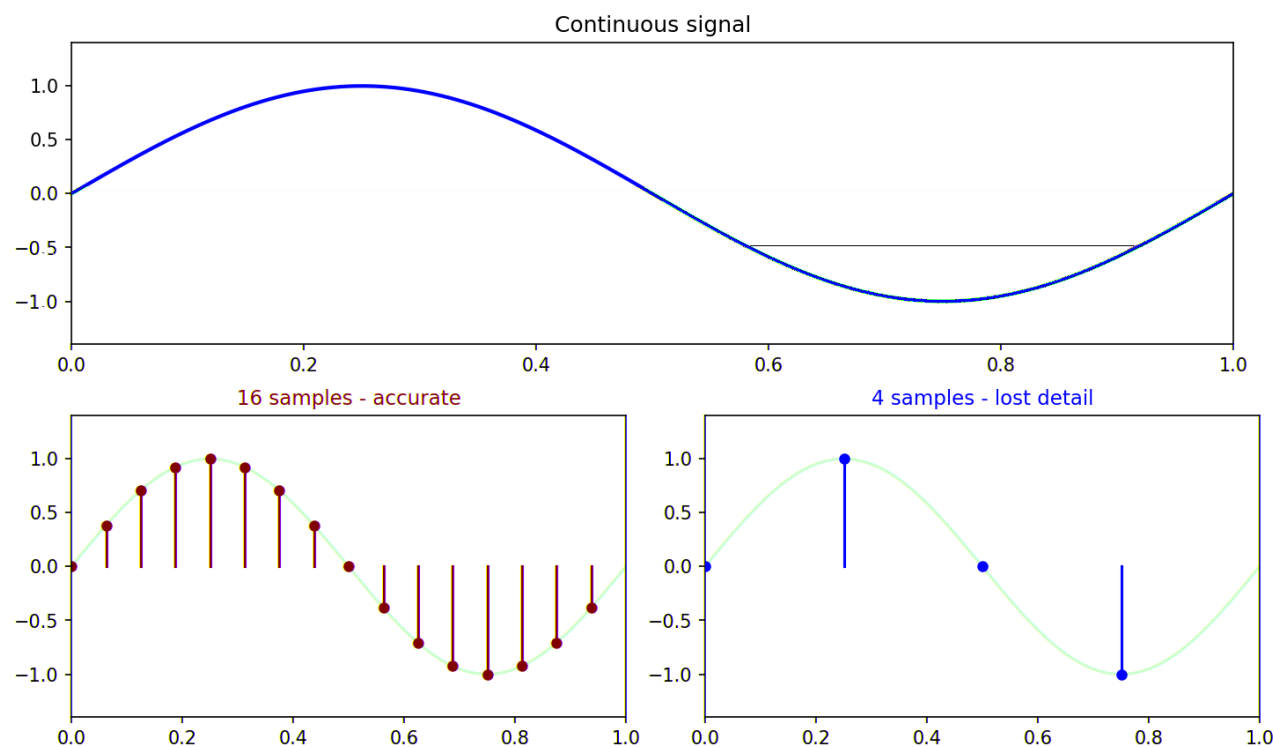
Златно правило: Променяй само по ЕДНО нещо наведнъж. Така разбираш какво причинява промяната.

Ка " а" "

Звукът в реалния свят е а а а а — въздухът вибрира плавно, без прекъсвания. Но компютърът работи само с числа. За да превърне вълна в числа, той прави две неща:

Са а а (sampling) — " а а а" а а

Представи си, че сигналът е морков.	Семплирането е нарязването му на парчета.
По-малки парчета (повече точки) = по-точна форма.	По-големи парчета (по-малко точки) = губиш детайли



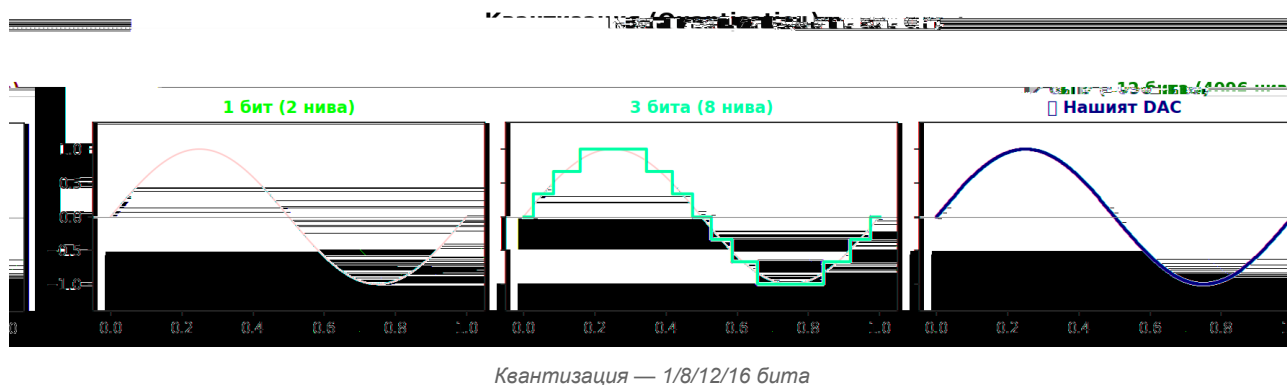
Sample rate = колко числа взимаш всяка секунда.

- Музиката обикновено се записва с **44100 Hz** (44 100 числа/сек)
- На нашата платка ползваме **22050 Hz** или по-малко — достатъчно за синтетични звуци

Квантизация (quantisation) — "апроксимация"

Всяко парче може да има определен брой нива. Колкото повече нива — толкова по-точно:

- 1 бит** = 2 нива (вкл/изкл) може само квадратна вълна
- 8 бита** = 256 нива грубо, но разпознаваемо
- 12 бита** = 4096 нива **а** **DAC** — достатъчно за чист звук
- 16 бита** = 65536 нива CD качество

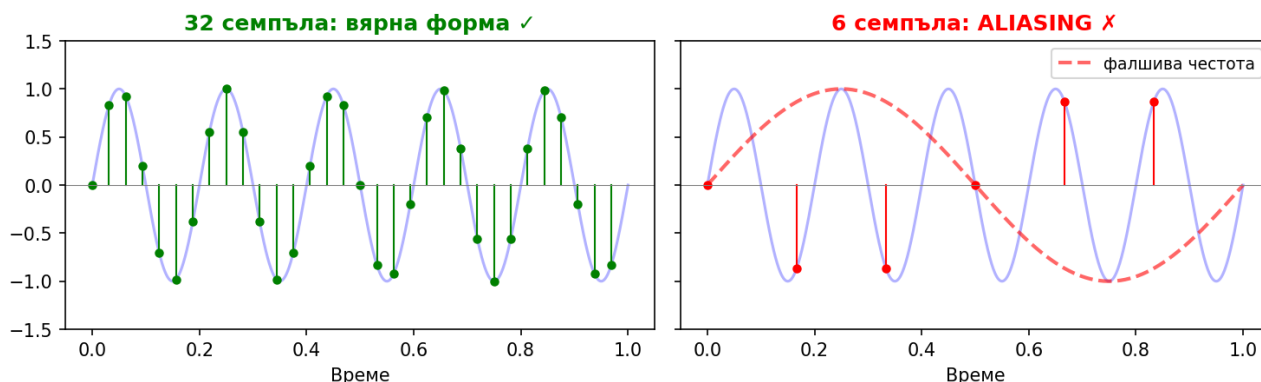


На **VK_RA4M2** DAC-ът е **12-б** : стойности от 0 до 4095.

Такава апроксимация — апроксимация

За да запазиш честота **f** в сигнала, трябва sample rate поне $2 \times f$.

Aliasing — фалшиви честоти при нисък sample rate



Aliasing — достатъчен vs нисък sample rate

- Човешкото ухо чува до ~20 000 Hz
- Затова CD quality е 44 100 Hz ($> 2 \times 20\,000$)
- Ако sample rate е **а** получаваш **aliasing**

Пример на RA4M2:

```
# Буфер от 64 точки, искаме нота 440 Hz:
sample_rate = 440 * 64 # = 28160 Hz
# Най-високата честота: 28160 / 2 = 14080 Hz

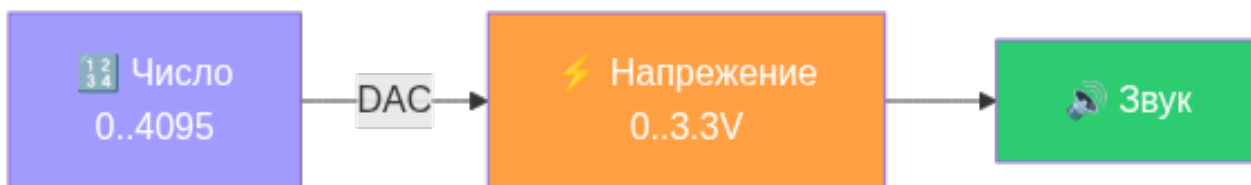
# Но ако буферът е само 4 точки:
sample_rate = 440 * 4 # = 1760 Hz
# Максимална честота: 880 Hz — тонът ще звучи ГРЕШНО (aliasing)
```

За :

- Повече точки на период = по-чист звук
- По-висок sample rate = по-високи честоти
- **12 бита** (4096 нива) е достатъчно за синтетични звуци

У 0 — П

А а : Представи си, че DAC-ът е кран за вода. Числото, което му даваш (0–4095), определя колко е отворен кранът. 2048 = наполовина.



DAC pipeline — число → напрежение → звук

К а : `machine.DAC, array('H'), MID = 2048, clamp12(), write_timed()`.

```
from machine import DAC, Pin
from array import array

MID = 2048 # Средна стойност (тишина)
dac = DAC(Pin("P014")) # Създаваме DAC на пин P014
dac.write(MID) # Тишина — кранът е наполовина

# Правим прост "бип" — триъгълна вълна от 32 числа
buf = array("H", [0] * 32) # Буфер с 32 числа (uint16)
for i in range(16):
    buf[i] = MID + i * 120 # Нагоре: 2048 → 3968
    buf[31 - i] = MID + i * 120 # Надолу (огледално)

dac.write_timed(buf, 440 * 32, mode=DAC.CIRCULAR, transfer=DAC.TRANSFER_DTC)
```

Ка а : нота Ла (440 Hz) — непрекъснат тон.

Е а а : Промени **440** на **880** — нотата скача с една октава!

Важно: array('H', ...) означава масив от беззнакови 16-битови числа. DAC приема САМО такъв формат. Стойностите са от 0 до 4095 (12 бита).

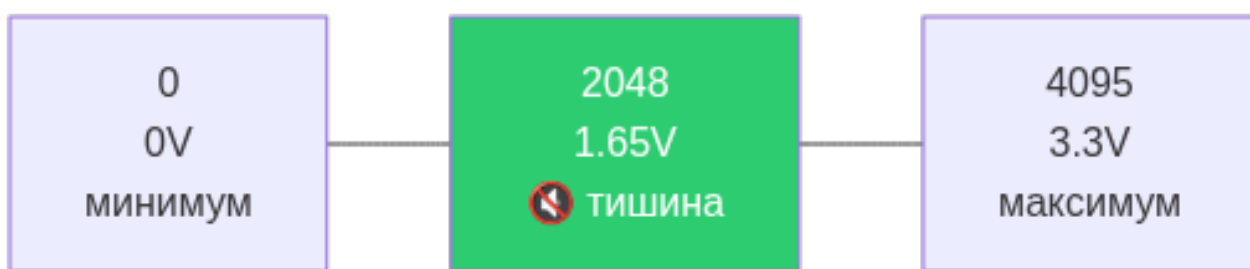
Ка а `clamp12()`? Функция, която не позволява числото да излезе извън 0–4095:

```
def clamp12(v):
    if v < 0: return 0
    if v > 4095: return 4095
    return v
```

Ще я ползваш навсякъде — запомни я!

У 1 — DC, AC а а а а

А а : DC е като езеро — водата стои на едно ниво. AC е като морски вълни — водата се качва и слиза около средното ниво.



DC/AC обхват: 0 → 2048 → 4095

Всеки звук е **аба а а а а**. Колкото по-голямо е колебанието (амплитудата), толкова по-силен е звукът.

```
from machine import DAC, Pin
dac = DAC(Pin("P014"))

dac.write(2048) # DC — тишина (средна точка)
dac.write(4000) # DC — високо напрежение, но НЕ е звук!
dac.write(100)  # DC — ниско напрежение, пак НЕ е звук!
```

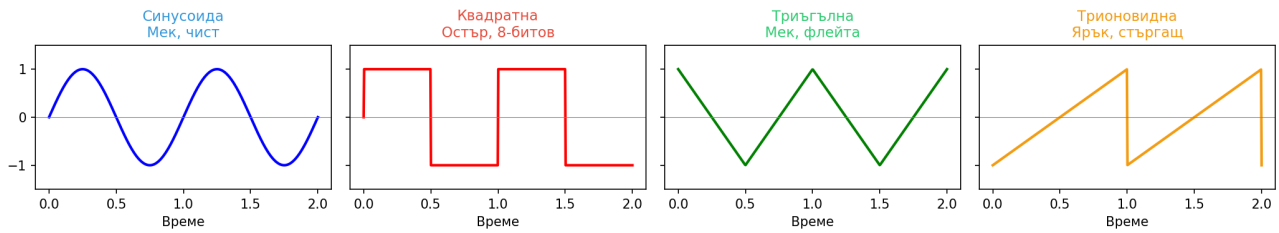
К а а : Звукът е ПРОМЯНА. Постоянна стойност = тишина.

Е а а : С осцилоскоп на P014, виж че `dac.write(2048)` дава равна линия.

У 2 — В а

А а : Различните форми на вълните звучат различно — като различни музикални инструменти.

Вълнови форми



Четири вълнови форми

Платката вече има готов модул `retro_synth.py` с всички вълнови форми!

```
from retro_synth import RetroSynth

synth = RetroSynth("P014")

# Чуй различните форми на една и съща нота (Ла, 440 Hz):
synth.play_note("A4", 500, "square")      # Квадратна – "чиптюн"
synth.play_note("A4", 500, "triangle")    # Триъгълна – "мека флейта"
synth.play_note("A4", 500, "saw")         # Трионовидна – "остър тембър"
synth.play_note("A4", 500, "pulse25")    # Пулс 25% – "носов звук"
synth.stop()
```

Ка **а** : 4 различни тембъра на ЕДНА и съща нота.

За **а** **а** ? Защото формата на вълната определя кои *обертони* (допълнителни честоти) са налични. Ще разбереш повече в Урок 9 (FFT).

У **3 — Ка** **а** **а** **а** **а** **а** **а**

А **а** : Рисуваш точки на милиметрова хартия и ги свързваш. Всяка точка е едно число в буфера. Колкото повече точки — толкова по-гладка е кривата.

К **а** : `wavetable`, `math.sin()`, `sample rate`, `период`.

```
from array import array
import math

MID = 2048                # Средна точка на DAC
AMP = 1400                # Амплитуда (колко силно)
N = 64                    # Точки на един период

buf = array("H", [0] * N) # Празен буфер
for i in range(N):
    angle = 2.0 * math.pi * i / N # Ъгъл: 0 → 2π
    value = MID + int(AMP * math.sin(angle))
    buf[i] = max(0, min(4095, value)) # clamp до 12 бита
```

Какво прави всяка стъпка:

1. `angle` — къде сме в кръга (0° 360°)
2. `math.sin(angle)` — дава число от -1.0 до +1.0

4. `clamp` — не позволява да излезем от 0–4095

Φ **a a:** sample rate = честота × брой точки на период

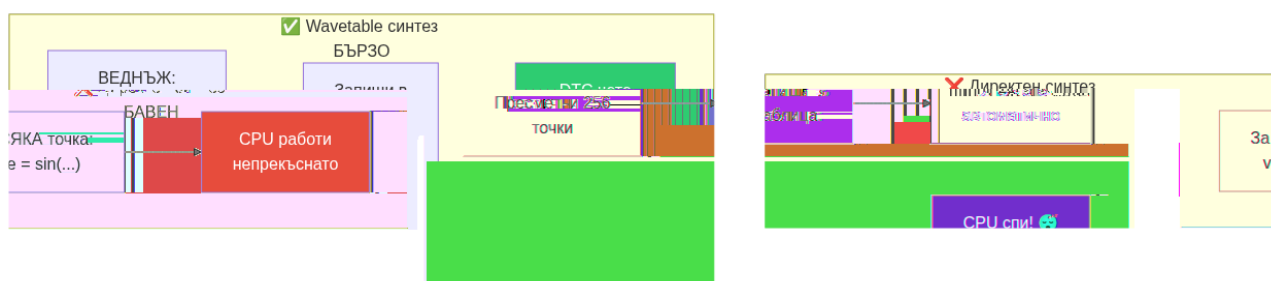
```
sample_rate = 440 * 64    # = 28160 точки/секунда

from machine import DAC, Pin
dac = DAC(Pin("P014"))
dac.write timed(buf, sample_rate, mode=DAC.CIRCULAR, transfer=DAC.TRANSFER_DTC)
```

DTC = хардуерен контролер, който подава точките на *DAC* автоматично. *CPU*-то е свободно. Максимум 256 точки за *DTC*. За повече — *DMAC*.

Е а а : Промени $N = 64$ на $N = 8$. Звукът става "грубо" — по-малко точки = по-ъглеста вълна.

Wavetable a — a a 6 ?



Wavetable синтез — директен vs lookup

RetroSynth прави точно това вътрешно — има готови wavetable-и за всяка форма.

П а а:

- **Б** — няма изчисления по време на възпроизвеждане
- **Г а** — смени таблицата = смени тембъра
- **Ма а а** — 256×2 байта = само 512 байта на вълна

4 — Signed vs Unsigned

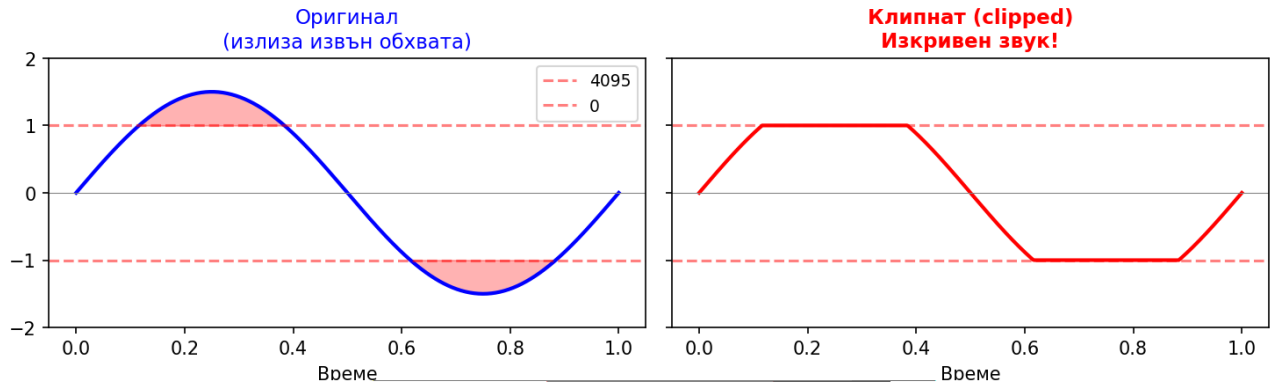
А а : DAC е като чаша — побира от 0 до 4095 "капки". Ако се опиташ да налееш 5000 — прелива.

Ф а	Об а	К а а а
Signed (със знак)	-1400 .. +1400	при изчисления (sin, mix)
Unsigned (без знак)	0 .. 4095	при подаване към DAC

Конверсия: $\text{unsigned} = \text{signed} + \text{MID}$ (MID = 2048)

K a a (Clipping)

Клипване (Clipping)



Клипване — оригинал vs clipped

```
# Ако два сигнала се съберат и резултатът е 5000:
value = signal_a + signal_b + MID    # = 5000
# DAC приема само 0..4095 → горната част се "отрязва"
```

Как да го избегнеш — **headroom** (запас):

```
# Намали амплитудата ПРЕДИ да ги събереш:
signal_a = int(original_a * 0.5)    # Половин сила
signal_b = int(original_b * 0.5)    # Половин сила
value = clamp12(signal_a + signal_b + MID)  # Вече не клипва!
```

Правило за палеца: Ако смесваш N сигнала, намали всеки по $1/N$.

Aliasing — а а а а

Ако sample rate е прекалено нисък — DAC-ът „вижда“ а а а а.

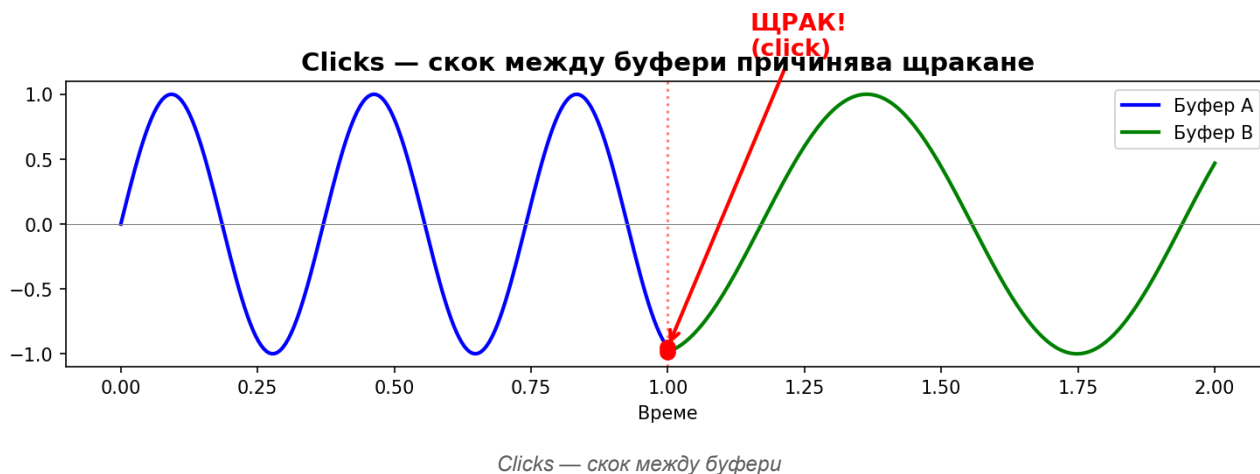
```
from array import array
import math
from machine import DAC, Pin

MID, AMP, N = 2048, 1400, 8          # Само 8 точки!
buf = array("H", [0] * N)
for i in range(N):
    # 3 периода в 8 точки — невъзможно!
    buf[i] = max(0, min(4095, MID + int(AMP * math.sin(2 * math.pi * 3 * i / N))))

dac = DAC(Pin("P014"))
dac.write_timed(buf, 8000, mode=DAC.CIRCULAR, transfer=DAC.TRANSFER_DTC)
# Чуваш ГРЕШЕН тон — aliasing!
```

Правило: Никога не слагай повече от $N/2$ периода в буфер от N точки (Найкуист).

Clicks (а а)



Как да ги избегнеш:

- В **CIRCULAR** режим: буферът трябва да е **1 а**
- При смяна: изчакай (**dac.stop()**) или fade out fade in
- **RetroSynth** се грижи за това автоматично

У 5 — И а а а: а а а ?

А а : Как измерваш колко силен е вятърът? Не гледаш само най-силния порив — измерваш *средната* сила. Така е и с RMS.

К а : `dsp.rms()`, `dsp.power()`, `array('f')`.

```
import dsp
from array import array
import math

N = 128
buf = array("f", [0.0] * N)

# Генерираме синусоида с амплитуда 1.0
for i in range(N):
    buf[i] = math.sin(2 * math.pi * i / N)

rms_val = dsp.rms(buf)      # ≈ 0.707 (= 1/√2 за синусоида)
pwr_val = dsp.power(buf)    # ≈ 64.0 (= N/2)

print("RMS:", rms_val)
print("Power:", pwr_val)
```

Ка а RMS?

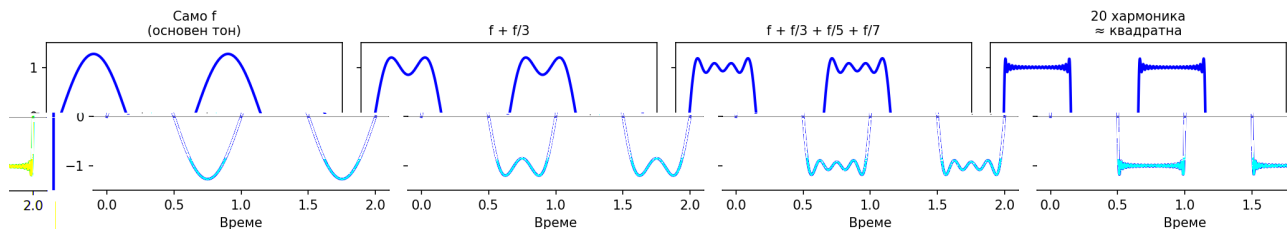
- **Root Mean Square** = Корен от Средното на Квадратите
- За синусоида с амплитуда A: **RMS = A / 2 0.707 × A**
- Показва "ефективната сила" на сигнала

Е а а : Промени амплитудата от 1.0 на 0.5 — RMS пада наполовина. Промени формата от sin на квадратна — RMS се качва.

Математикът Жозеф Фурие е доказал нещо удивително:

Всеки периодичен сигнал може да се разложи на сума от синусоиди.

Теорема на Фурие — квадратна вълна от синусоиди



Теорема на Фурие — хармоници → квадратна вълна

- Квадратната вълна = синусоида + нечетни хармоници (3×, 5×, 7×...)
- Трионовидната вълна = синусоида + ВСИЧКИ хармоници
- Звукът на пиано = основна честота + много хармоници с различна сила

```
from array import array
import math

MID = 2048
N = 256
buf = array("H", [0] * N)

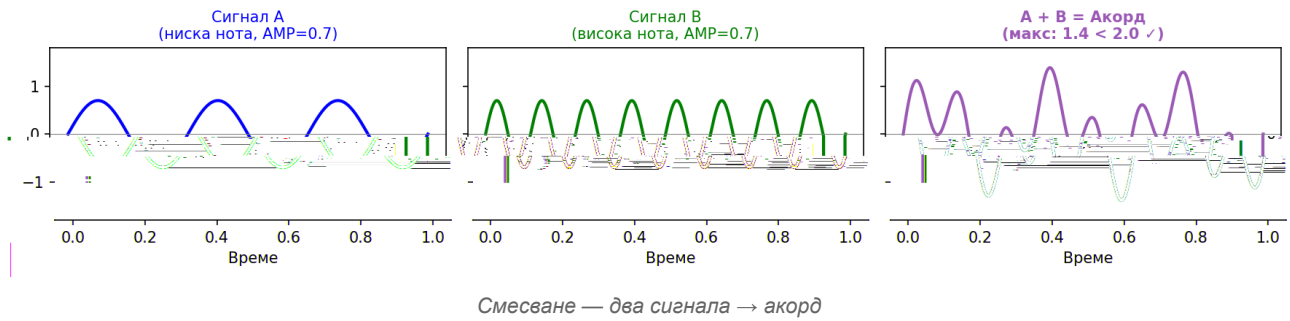
for i in range(N):
    # Основен тон + 3-ти + 5-ти хармоник
    val = math.sin(2 * math.pi * i / N)          # f
    val += (1/3) * math.sin(2 * math.pi * 3 * i / N) # 3f
    val += (1/5) * math.sin(2 * math.pi * 5 * i / N) # 5f
    buf[i] = max(0, min(4095, MID + int(900 * val)))
```

Ка а : С 3 хармоника звучи вече „квадратно“. Добави 7-ми и 9-ти — звукът става все по-остър.

В У 9 (FFT) ще видиш обратния процес — как да „разгловиш“ звук на честоти.

А а : Смесване на звук е като смесване на бои. Ако смесиш синьо и жълто — получаваш зелено. Ако смесиш две ноти — получаваш акорд.

Смесване на два сигнала (адитивен синтез)



```
from array import array
import math
from machine import DAC, Pin

MID = 2048
AMP = 700 # Половин амплитуда (700 × 2 = 1400 < 2048)
N = 128
FREQ_A = 3 # 3 периода в буфера (ниска нота)
FREQ_B = 5 # 5 периода в буфера (висока нота)

buf = array("H", [0] * N)
for i in range(N):
    a = AMP * math.sin(2 * math.pi * FREQ_A * i / N)
    b = AMP * math.sin(2 * math.pi * FREQ_B * i / N)
    value = int(a + b) + MID
    buf[i] = max(0, min(4095, value))

dac = DAC(Pin("P014"))
dac.write_timed(buf, 22050, mode=DAC.CIRCULAR, transfer=DAC.TRANSFER_DTC)
```

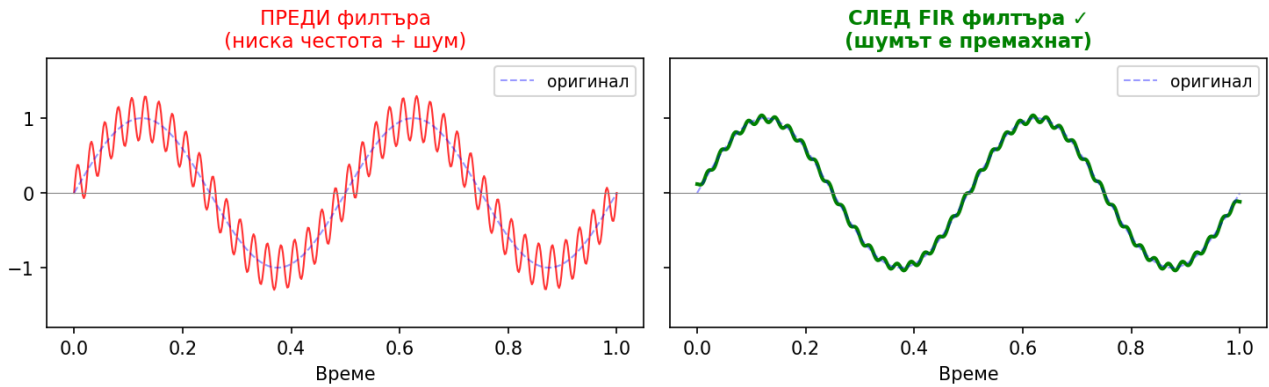
За AMP а 700, а 1400? Два сигнала по 700 макс 1400 (< 2048). По 1400 2800 > 2048 клипване!

Е а а : Промени AMP на 1400 — чуй изкривяването. После на 700 — чист звук.

У 7 — Ф а а: FIR (а а)

А а : Филтърът е като цедка за чай. Пропуска течността (ниски честоти), но задържа листата (високи честоти).

FIR нискочестотен филтър



FIR филтър — преди и след

Как работи FIR:

- Взима няколко последователни числа от сигнала
- Умножава всяко по тежестен коефициент
- Сумира едно изходно число
- Сякаш правиш "плъзгащо средно" (moving average)

```
import dsp
from array import array
import math

N = 128
BLOCK = 64

# Входен сигнал: ниска честота + високочестотен шум
src = array("f", [0.0] * N)
for i in range(N):
    src[i] = math.sin(2 * math.pi * 2 * i / N)      # Ниска
    src[i] += 0.3 * math.sin(2 * math.pi * 20 * i / N)  # Шум

# FIR коефициенти — 5-tap moving average (обърнати за CMSIS-DSP)
coeffs = array("f", [0.2, 0.2, 0.2, 0.2, 0.2])

filt = dsp.FIRFilter(coeffs, BLOCK)
dst = array("f", [0.0] * N)
filt.process(src, dst)

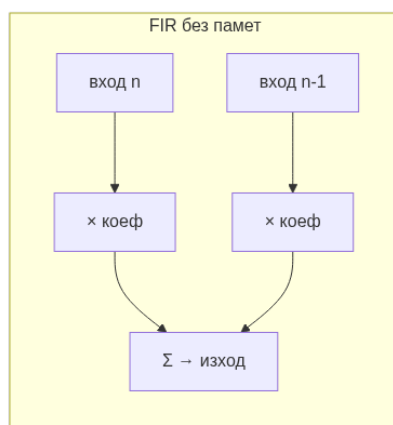
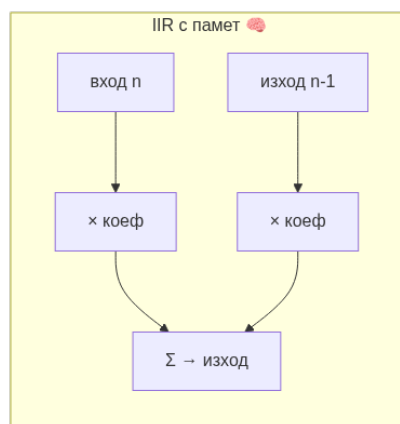
print("Before:", src[10])    # С шум
print("After:", dst[10])     # Изгладен
```

Е а а : Промени коефициентите на `[0.1, 0.2, 0.4, 0.2, 0.1]` — по-добър филтър (повече тежест в средата).

У 8 — Ф а а: IIR (biquad)

А а : FIR е като нов филтър — всеки път гледа само входните данни. IIR е като филтър с

"памет" — използва и предишните си собствени изходи. Затова е по-ефективен.



FIR vs IIR блок-схема

С	FIR	IIR
Памет	Няма (само вход)	Да (вход + изход)
Коефициенти	Много (напр. 32)	Малко (5 на stage)
Стабилност	Винаги стабилен	Може да "избухне"

```

import dsp
from array import array
import math

N = 128
BLOCK = 64

# Входен сигнал: ниска + висока честота
src = array("f", [0.0] * N)
for i in range(N):
    src[i] = math.sin(2 * math.pi * 2 * i / N)
    src[i] += 0.5 * math.sin(2 * math.pi * 30 * i / N)

# Biquad коефициенти: {b0, b1, b2, -a1, -a2}
# ВНИМАНИЕ: a1 и a2 са с ОБЪРНАТ знак спрямо учебниците!
coeffs = array("f", [0.07, 0.14, 0.07, 1.27, -0.55])

filt = dsp.IIRFilter(coeffs, 1, BLOCK) # 1 stage, блок 64
dst = array("f", [0.0] * N)
filt.process(src, dst)

print("Before:", src[10])
print("After:", dst[10])

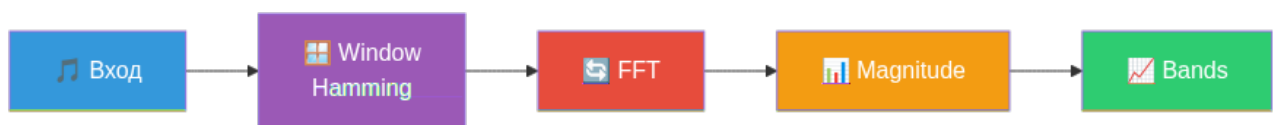
```

Внимание: IIR коефициентите за a1 и a2 трябва да бъдат с обрнат знак спрямо стандартните формули. Ако в учебника пише a1 = -1.27, тук записваш +1.27.

Е а а : Промени 0.55 на 0.95 — филтърът става по-тесен, но може да стане нестабилен.

У 9 — FFT: а а а а а ?

А а : Ако чуеш оркестър, FFT е като да разпознаеш кой инструмент свири коя нота — разделя звука на отделните му честоти.



FFT pipeline

FFT pipeline — стъпка по стъпка:

1. **П а** (window) — "изглажда" краищата на буфера
2. **FFT** — трансформира от време честота
3. **Ма а** — колко силна е всяка честота
4. **Ла** (bands) — групира честотите в удобни блокове

```

import dsp
from array import array
import math

N = 128
HALF = N // 2 + 1    # = 65

# Генерираме сигнал с две честоти
src = array("f", [0.0] * N)
for i in range(N):
    src[i] = math.sin(2 * math.pi * 5 * i / N)          # Честота 5
    src[i] += 0.5 * math.sin(2 * math.pi * 20 * i / N)  # Честота 20

fft = dsp.FFT(N)

win = array("f", [0.0] * N)      # Буфер за прозорец
cx = array("f", [0.0] * N)       # Буфер за комплексен резултат
mag = array("f", [0.0] * HALF)   # Буфер за магнитуди

fft.window_apply(src, win)        # Стъпка 1: Hamming прозорец
fft.run(win, cx)                  # Стъпка 2: FFT трансформация
fft.magnitude(cx, mag)            # Стъпка 3: комплексно → магнитуда

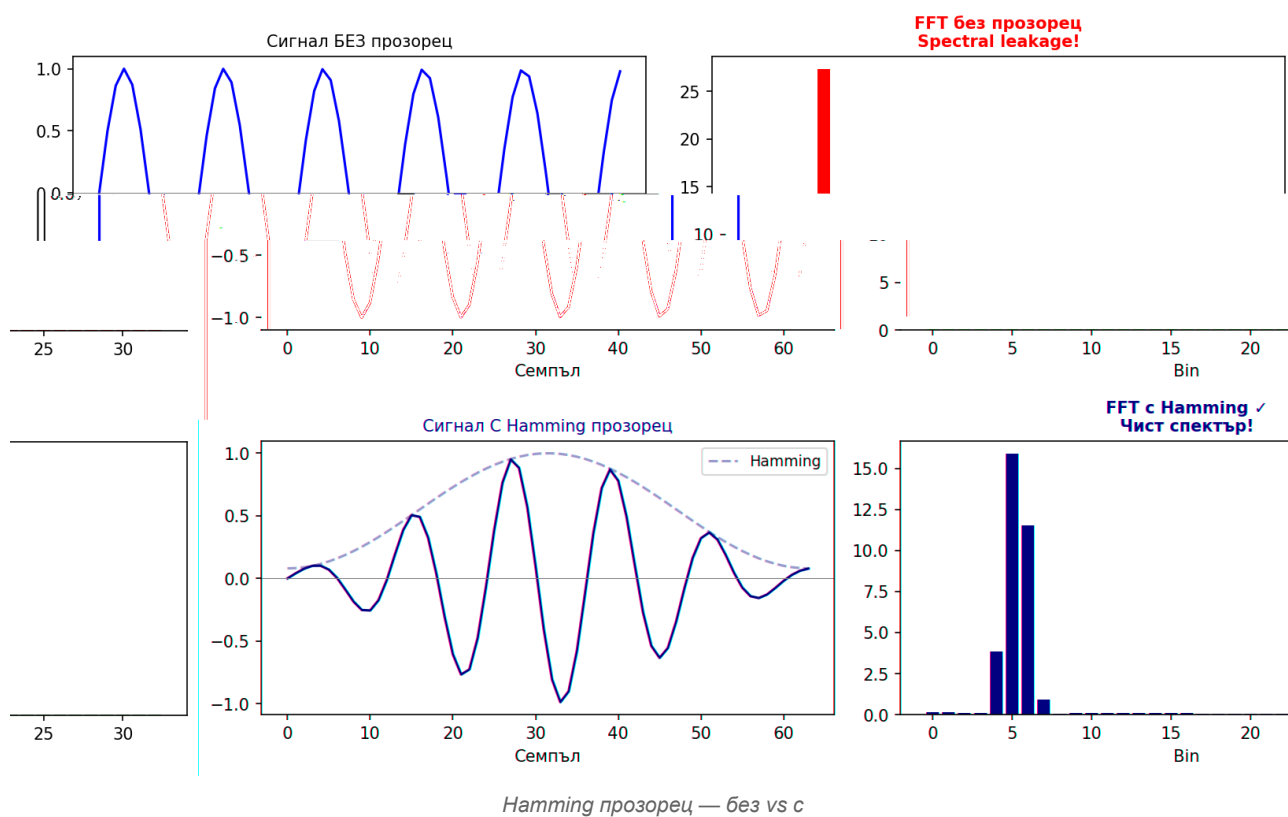
# Намираме пиковите
for i in range(HALF):
    if mag[i] > 0.1:
        print("Bin", i, ":", mag[i])

```

Ка **а** ? Два пика — при bin 5 и bin 20. Точно честотите, които сложихме!

Ка **а Hamming** **а** ? Умножава сигнала по звънчевидна крива, за да няма "скок" в краищата на буфера. Без него FFT показва фалшиви честоти (spectral leakage).

Hamming прозорец — защо е нужен?



Е а а : Подай `src` директно на `fft.run` без `window_apply`. Виж допълнителните фалшиви пикове.

У 10 — Audiomixer: а аа а а

А а : В Урок 6 миксирахме два сигнала предварително в буфер. Но `audiomixer` е като DJ пулт — смесва няколко звука **а**, в реално време.


```

import audiomixer
from array import array
import math
from machine import Pin

N = 128
tone_lo = array("h", [0] * N)    # 'h' = signed 16-bit
tone_hi = array("h", [0] * N)

for i in range(N):
    tone_lo[i] = int(16000 * math.sin(2 * math.pi * 3 * i / N))
    tone_hi[i] = int(16000 * math.sin(2 * math.pi * 8 * i / N))

# Създаваме миксер: 2 гласа, 22050 Hz sample rate
mixer = audiomixer.Mixer(2, 22050, pin=Pin("P014"))

mixer.voice[0].level = 1.0        # Глас 0: пълна сила
mixer.voice[0].play(tone_lo, repeat=True)

mixer.voice[1].level = 0.3        # Глас 1: 30% сила
mixer.voice[1].play(tone_hi, repeat=True)

# Промени level в реално време от REPL:
# mixer.voice[1].level = 0.8

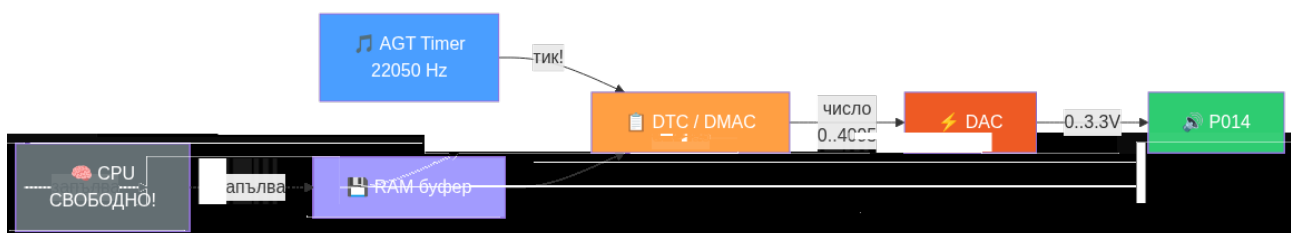
```

С	Р а (У 6)	audiomixer (У 10)
Кога се смесва	Предварително	В реално време
Промяна на звук	Спри пресметни пусни	Промени level и готово
Трансфер	DTC (256 семпла)	DMAC double-buffer

Е а а : Докато миксерът свири, промени `mixer.voice[1].level` от REPL. Звукът се променя мигновено!

Ха а а: Timer DTC/DMAC DAC

Как платката възпроизвежда звук без да натоварва процесора?



AGT → DTC/DMAC → DAC pipeline

CPU не участва в нито една стъпка! Затова можеш да правиш друго докато звукът свири.

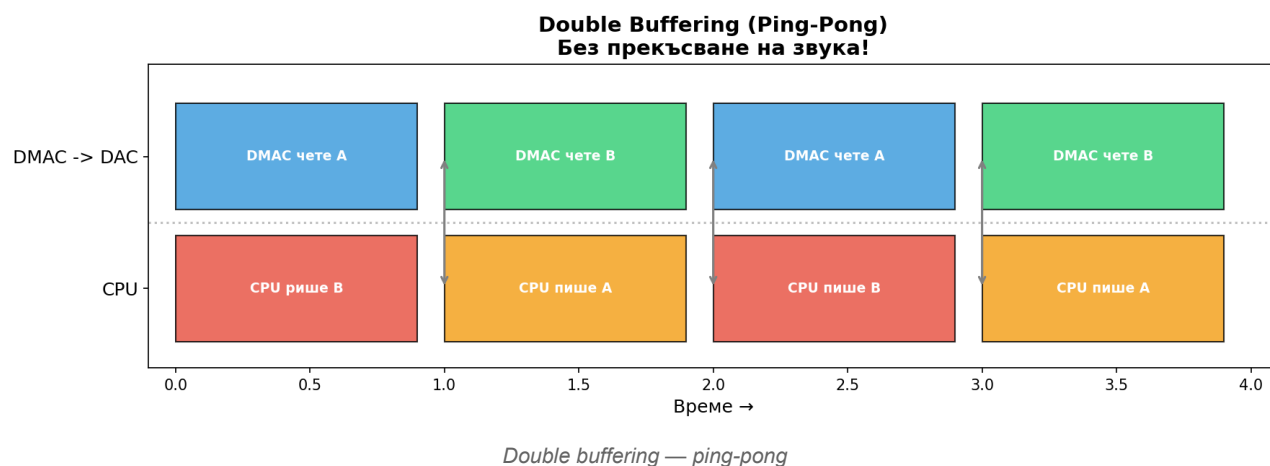
DTC vs DMAC

С	DTC	DMAC
Максимален буфер	256 семпла	65536 семпла

Циклично (CIRCULAR)	Да	Да (с double buffer)
Double buffering	Не	Да — ping-pong
Кога го ползваш	Кратки wavetable цикли	audiomixer, дълги ефекти

Double Buffering (Ping-Pong)

Проблем: ако имаш **а** буфер, докато DMAC го чете, ти не можеш да го промениш щракания.



audiomixer прави точно това автоматично — затова можеш да сменяш **level** и да пускаш нови гласове без щракания.

У 11 — П а : С а а а а а а LED а а

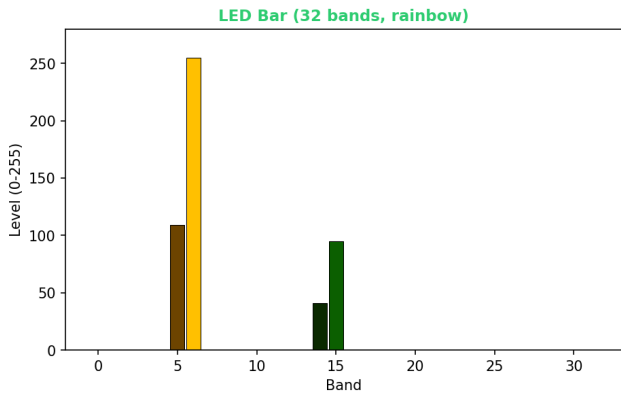
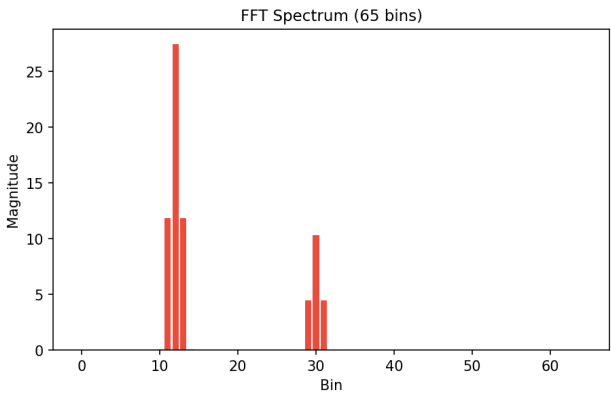
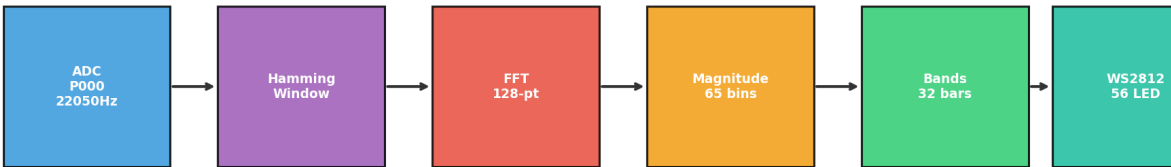
А а : Представи си, че гледаш еквалайзера на стерео уредба — цветните колони скачат нагоре-надолу в ритъма на музиката. Точно това ще направим, но на нашата платка!

Този проект е **а а** а на курса — събира всичко, което научи:

- **FFT** (Урок 9) — разделя звука на честоти
- **Ла (bands)** — групира 65 FFT бина в 32 цветни колони
- **Peak hold** — пикът пада бавно, за красив ефект
- **WS2812** — адресируема RGB LED лента показва резултата

DSP Pipeline

Spectrum Analyzer - DSP Pipeline



Spectrum Analyzer — DSP Pipeline + LED визуализация

Пътят на сигнала минава през 6 , всички на хардуерно ниво:

С а	Ка а	В а а
1. `AudioADC`	Семплира микрофона: 128 числа × 22050 Hz	~5.8 ms
2. `window_apply()`	Hamming прозорец (без него leakage)	<1 ms
3. `fft.run()`	128-точкова RFFT (CMSIS-DSP, бърза)	~35 μs
4. `magnitude()`	Комплексни → реални магнитуди (65 бина)	<1 ms
5. `bands()`	65 бина → 32 логаритмични ленти (0–255)	<1 ms
6. WS2812 `write_async()`	Изпраща цветовете на 56 LED	~2 ms

Об : ~10 ms на кадър = ~100 кадъра/секунда!

Ха а а а а



Ка аб

1. AudioADC семплира с DTC ping-pong (два буфера):

```
adc = AudioADC(Pin("P000"), sample_rate=22050, frame_len=128)
adc.start()
# Когато кадър е готов:
adc.read_f32(frame_buf)    # int16 → float32 [-1.0 .. +1.0]
```

2. FFT + Bands:

```
fft.window_apply(frame_buf, fft_buf)    # Hamming прозорец
fft.run(fft_buf, fft_buf)               # 128-pt RFFT
fft.magnitude(fft_buf, mag_buf)         # → 65 магнитуди
fft.bands(mag_buf, band_buf)            # → 32 ленти (0-255)
```

3. Peak hold — пикът пада бавно:

```
for i in range(N_BANDS):
    if band_buf[i] >= peak_buf[i]:
        peak_buf[i] = band_buf[i]      # Нов пик!
    else:
        peak_buf[i] -= DECAF           # Бавно спускане
```

4. Rainbow render — всяка лента получава цвят по HSV:

```
for i in range(N_BANDS):
    hue = (i * 170) // N_BANDS         # Червено → синьо
    brightness = peak_buf[i]
    hsv_to_rgb(hue, brightness) → pixel_buf
```

5 a a a

Истинският `spectrum_analyzer.py` поддържа 5 режима, които се сменят с бутон:

#	Pa	Ка а
0	Rainbow	Всяка лента е различен цвят (червеносиньо)
1	TriBar	3 цветни зони: ниски=зелено, средни=жълто, високи=
2	Center	Лентите растат от центъра навън
3	MSGEQ7	7 широки ленти (класически еквайзер)
4	Pulse	Целият стрип пулсира с RMS на сигнала

Та а а — БЕЗ

Нямаш микрофон? Не е проблем! Файлът `spectrum_analyzer_test.py` заменя `AudioADC` с а а а а а — синусоида, чиято честота плавно се покачва от 100 Hz до 10000 Hz и обратно (sweep).

Ка б а а : Светещ „пик“ който бавно пълзи от ляво (ниски честоти) към дясно (високи честоти) и обратно. Всеки цикъл отнема ~6 секунди.

Как работи тестът:

```
# Вместо adc.read_f32(frame_buf) генерираме сигнал софтуерно:
def generate_test_signal(freq_hz, amplitude=0.8):
    for i in range(FFT_LEN):
        frame_buf[i] = amplitude * math.sin(2.0 * math.pi * freq_hz * i / FS_HZ)

# Sweep: логаритмично 100 Hz → 10000 Hz (60 стъпки)
for s in range(SWEEP_STEPS):
    ratio = s / (SWEEP_STEPS - 1)
    freq = 100 * (10000/100) ** ratio    # Log sweep
    generate_test_signal(freq)
# ... FFT pipeline + WS2812 render (същият код)
```

За да го пуснеш:

```
mpremote run spectrum_analyzer_test.py
```

Очакван изход в REPL:

```
=== Spectrum Analyzer TEST (no microphone) ===
WS2812: P112 pixels: 56
Sweep: 100 Hz -> 10000 Hz, 60 steps
Ctrl+C to stop.

UP    100 Hz  band= 0  val=255
UP    422 Hz  band= 3  val=248
UP    1778 Hz band=11  val=251
UP    7499 Hz band=24  val=244
DOWN  7499 Hz band=24  val=232
DOWN  1778 Hz band=11  val=239
DOWN  422 Hz  band= 3  val=241
--- Cycle 1 complete ---
```

Ка а а а :

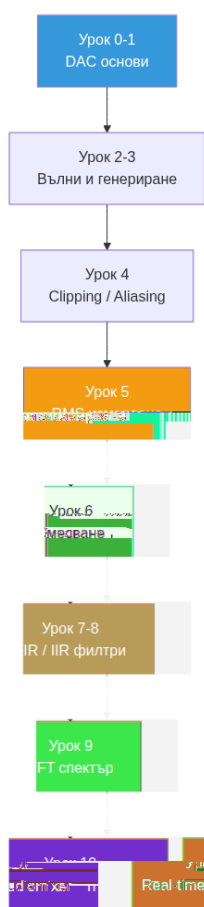
К а	П а ?
`dsp.FFT` — window, run, magnitude	
`fft.bands()` — log mapping	
Peak hold c decay	
HSV RGB render	
WS2812 async write	
Zero-allocation main loop	

Е а а :

- Промени **FREQ_MAX** на **5000** — sweep-ът ще покрие само лявата половина
- Промени **DECAY** на **2** — пиковите ще падат бавно (дълга „опашка“)
- Добави втора синусоида в **generate_test_signal()** — ще видиш 2 пика едновременно

Об б а а — а а а

У	Та а	И а	Ка а
0	Първи звук	`DAC.write_timed()`	Подава числа звук
1	DC / AC / midpoint	`DAC.write()`	Разбираш тишина vs звук
2	Вълнови форми	`RetroSynth`	Чуваш square, triangle, saw
3	Генериране с числа	`math.sin()` + `array`	Създаваш вълна от нулата
4	Signed / clipping	`clamp12()`	Предпазваш от изкривяване
5	Измерване	`dsp.rms()`	Измерваш силата на сигнала
6	Смесване	`+` оператор	Събираш два сигнала в един
7	FIR филтър	`dsp.FIRFilter`	Премахваш високи честоти
8	IIR филтър	`dsp.IIRFilter`	Компактен филтър с памет
9	FFT спектър	`dsp.FFT`	Виждаш от какви честоти е звукът
10	Real-time mixer	`audiomixer.Mixer`	Смесваш гласове в реално време
11	Спектрален анализатор	FFT + WS2812	Проект: визуализация на звук!



Път на обучение

Г а а а а а а

Тези файлове вече съществуват в [examples/](#) и можеш да ги пуснеш:

Фа	Ка а
`retro_synth.py`	Помощен модул: вълни, ноти, SFX (coin, jump, laser)
`dac_retro_sfx.py`	Демо на звукови ефекти (Super Mario стил)

`dac_retro_music.py`	Мелодия + арпеджо демо
`audiomixer_demo.py`	Два DAC канала с различни вълни
`spectrum_analyzer.py`	FFT спектрален анализатор с WS2812 LED лента
`spectrum_analyzer_test.py`	Тестов вариант БЕЗ микрофон (sweep синусоида)
`test_dac_dmac_dtc.py`	Тест на DTC и DMAC трансферни пътища
`dac_firmware_probe.py`	Валидация на firmware (DTC circular, one-shot, DMA

За да пуснеш пример:

```
mpremote cp retro_synth.py :flash/examples/retro_synth.py
mpremote run dac_retro_sfx.py
```

C a — API a a

`dsp` (CMSIS-DSP б a)

Ф / К а	О а а
`dsp.rms(buf)`	RMS на `array('f')` float
`dsp.power(buf)`	Сума на квадратите float
`dsp.FIRFilter(coeffs, block)`	FIR филтър. `.process(src, dst)`
`dsp.IIRFilter(coeffs, stages, block)`	IIR biquad филтър. `.process(src, dst)`
`dsp.FFT(N)`	FFT обект (N = 32..4096, степен на 2)
`fft.window_apply(src, dst)`	Hamming прозорец
`fft.run(src, dst)`	Forward Real FFT
`fft.magnitude(cx, mag)`	Комплексен магнитуда (N/2+1)
`fft.set_bands(n, db_floor)`	Настройка на N логаритмични ленти
`fft.set_bands_msgeq7(fs, db_floor)`	7 ленти по MSGEQ7 стандарт
`fft.bands(mag, out)`	Магнитуди 0-255 по лента

`audiomixer`

Ф / К а	О а а
`audiomixer.Mixer(voices, sr, pin=...)`	Създава миксер (1-16 гласа, DMAC)
`mixer.voice[i].play(buf, repeat=True)`	Пуска буфер на глас i